

EE-2003

Computer Organization and Assembly Language

Week 14 – Lecture 27

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Screen Location Calculation
- 3 String Printing at Desired Location

Quick Review: What was in the Last week?

- Display Memory
- Declaring Characters and Strings in Assembly
- Number Printing Using Div Algorithm

Introduction

- Previous algorithms used fixed attributes and fixed screen locations.
- Now we generalize to display text at **any screen position** with **any attribute**.
- Screen uses a 2D coordinate system (row, column).
- Video memory is linear (1D), so we must convert coordinates.

Mapping Coordinates to Memory

- Text mode has **80 columns** per row.
- Each character cell uses **2 bytes** (character + attribute).
- Formula:

$$\text{location} = (y \times 80 + x) \times 2$$

- The 8088 **MUL** instruction is used for multiplication.

MUL Instruction

- Performs an **unsigned** multiplication.
- If operand is 8-bit:

$$AL \times src \rightarrow AX$$

- If operand is 16-bit:

$$AX \times src \rightarrow DX : AX$$

- Used here to compute screen offset from row/column.

Overview of Modified String Printing

- Program now takes:
 - x-position
 - y-position
 - attribute
 - address of string
 - string length
- Screen location is calculated using $MUL + \text{shifts}$.
- String printed starting at the computed memory offset.

Program Code (Part 1)

```
[org 0x0100]
jmp start
message: db 'hello world'
length: dw 11

; print string at desired location
printstr: push bp
mov bp, sp
push es
push ax
push cx
push si
push di

mov ax, 0xb800
mov es, ax          ; video base
mov al, 80          ; columns per row
mull byte [bp+10]   ; multiply with y position
add ax, [bp+12]     ; add x position
```

Program Code (Part 2)

```
shl ax, 1           ; multiply by 2 (character cell = 2 bytes)
mov di, ax          ; point DI to screen location

mov si, [bp+6]      ; address of string
mov cx, [bp+4]      ; string length
mov ah, [bp+8]      ; attribute

nextchar:
    mov al, [si]     ; next character
    mov [es:di], ax  ; write char + attribute
    add di, 2
    add si, 1
    loop nextchar

pop di
pop si
pop cx
pop ax
pop es
pop bp
ret 10
```

Program Code (Main Program)

```
start:
    call clrscr

    mov ax, 30
    push ax          ; x-position
    mov ax, 20
    push ax          ; y-position
    mov ax, 1
    push ax          ; attribute (blue on black)

    mov ax, message
    push ax
    push word [length]

    call printstr

    mov ax, 0x4c00
    int 0x21
```

- Shifting was used for multiplying by 2 (screen cell = 2 bytes).
- 5 parameters → subroutine ends with **ret 10**.
- Example prints “hello world” at (30, 20) in blue on black.
- Changing x, y, or attribute changes the screen output accordingly.

Thank You!

Any Questions?

- Today we learned how screen coordinates map to linear video memory.
- The formula $(y \times 80 + x) \times 2$ allowed us to compute exact byte offsets.
- The MUL instruction simplified multiplying row and column values.
- Our updated subroutine printed strings at any position with any attribute.
- Passing parameters through the stack made the routine flexible and reusable.

“Assembly reveals how simple arithmetic and memory access turn coordinates into visible output.”